

VULNHGNN

Graph Based Vulnerability Detection, Localization & Self Healing for C/C++

COMPLETE TECHNICAL PROJECT DOCUMENTATION

Document OverView

Field	Value
Project	VulnHGNN
Title	Graph Based Vulnerability Detection, Localization, and Self Healing for C/C++
Domain	Software Security / Graph Neural Networks
Tech Stack	Python , Network X + PyTorch Geometric
Compiler / IR	clang → LLVM/IR
Interfaces	Flask Web UI + CLI
Emissions tracking	CodeCarbon

Software Security • Graph Neural Networks • LLVM/IR • PyTorch Geometric

1. Executive Summary :-

→ VulnHGNN is a Python based static vulnerability analysis system for C/C++ that combines compiler derived program structure, heterogeneous graphs, and a multi label Graph Neural Network. The documented pipeline compiles source code to LLVM IR, parses the IR into JSON, builds heterogeneous function graphs, performs per function inference with sigmoid probabilities and per class thresholds, and aggregates results at file level. Localization and a rule based healing/feedback loop are part of the project architecture; the supplied analysis identifies several of those modules by name while noting that their complete contents were not indexed.

- **Input:** C/C++ source files.
- **Compilation:** clang with -O0 and debug information; the pipeline also uses -disable-O0-optnone.
- **Intermediate representation:** LLVM IR followed by structured JSON.
- **Graph representation:** basic blocks and instructions with containment, control flow, and data flow relationships.
- **Model:** VulnGAT, a multi label GNN using PyTorch Geometric; the exact internal layer architecture is not available in the supplied index.
- **Decision logic:** sigmoid probabilities with per class thresholds; CWE-476 uses 0.35 while other observed classes use 0.5.
- **File level aggregation:** maximum probability over analyzed functions.
- **Interfaces:** Flask web UI and command-line pipeline.
- **Operational extras:** structured error dictionaries, failed compilation logging, IR validation, and CodeCarbon based emissions tracking.

2. Abstract :-

→ Automated vulnerability detection and repair in C/C++ is challenging because useful evidence is distributed across program structure rather than isolated text patterns. VulnHGNN addresses this problem by lowering source code into LLVM IR and representing functions as heterogeneous graphs containing basic blocks and instruction nodes connected through containment, control flow, and data-flow relationships. A GAT style multi label model, VulnGAT, produces CWE probabilities. The documented inference stage applies per-class thresholds and aggregates function level predictions to a file level result. Localization and rule based healing follow inference as part of the broader architecture, while a feedback loop is intended to revalidate patched code. Quantitative performance claims are deliberately excluded because benchmark results were not verifiable from the supplied repository content.

3. System Architecture Diagram :-

→ The architecture is a staged transformation pipeline. Each stage converts the output of the previous stage into a representation suitable for structural analysis or machine learning.

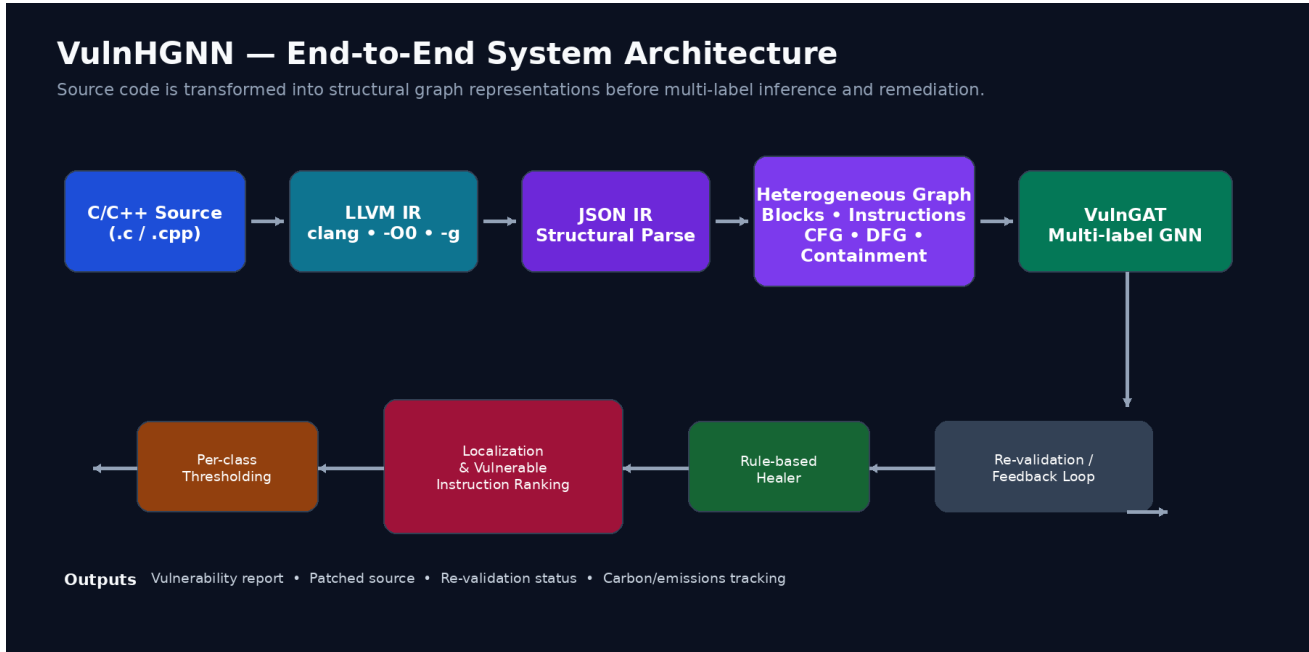


Figure 1. conceptual architecture diagram based on the supplied project description.

3.1 Architectural Layers

Layer	Responsibility	Primary artifact
Interface	Accept source file / invoke pipeline	Flask UI or CLI request
Compilation	Lower C/C++ into LLVM IR	*.ll
Parsing	Convert IR into structured JSON	IR JSON
Graph construction	Create heterogeneous graph	NetworkX / PyG graph
Inference	Run VulnGAT	Sigmoid probabilities
Decision	Apply per class thresholds	Predicted CWE labels
Localization	Rank/identify vulnerable regions	Localized findings
Healing	Apply rule based remediation	Patched source
Validation	Re-run analysis / verify result	Revalidation status
Reporting	Expose results and operational metadata	Vulnerability report

4. Workflow Diagram:-

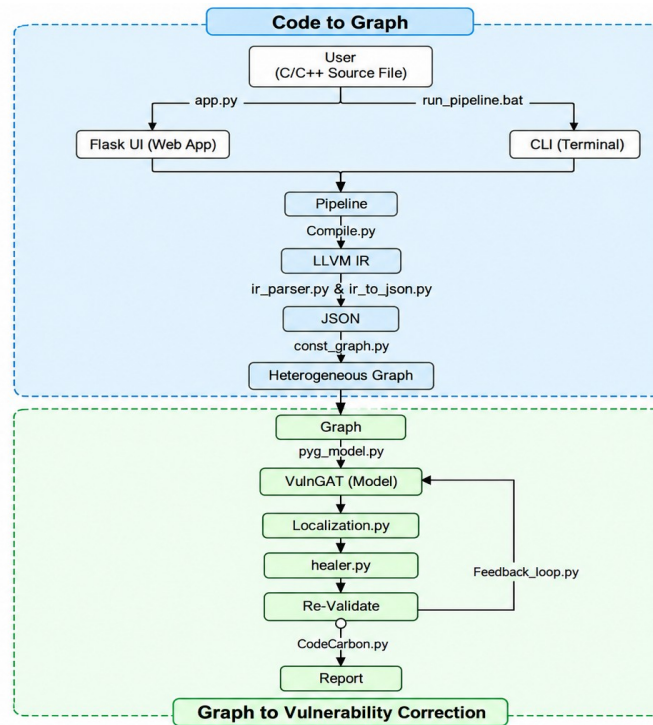


Figure 2. Supplied project pipeline diagram: source → LLVM IR → JSON IR → heterogeneous graph → model → thresholds → report → patches/re-validation.

4.1 Workflow Steps :-

- 1. Source ingestion:** The user provides a C/C++ source file through the Flask interface or CLI.
- 2. Compilation:** compile_to_ir invokes clang & writes LLVM IR into the documented data/ir location.
- 3. IR parsing:** The LLVM IR is transformed into structured JSON.
- 4. Graph construction:** Functions are represented using block/instruction nodes & structural edges.
- 5. Inference:** VulnGAT receives batched graph tensors and produces multi label logits/probabilities.
- 6. Thresholding:** Sigmoid probabilities are compared with per class thresholds.
- 7. Aggregation:** Function level predictions are combined into a file level verdict using the maximum probability.
- 8. Localization:** The architecture then localizes vulnerable code regions; the supplied analysis does not expose the exact localization equations.
- 9. Healing:** A rule based healer attempts a patch.

10. Re-validation: The feedback loop re-runs analysis to verify the patched result.

11. Reporting: The system returns a structured result/report and may track carbon emissions.

5. Sequence Diagram :-

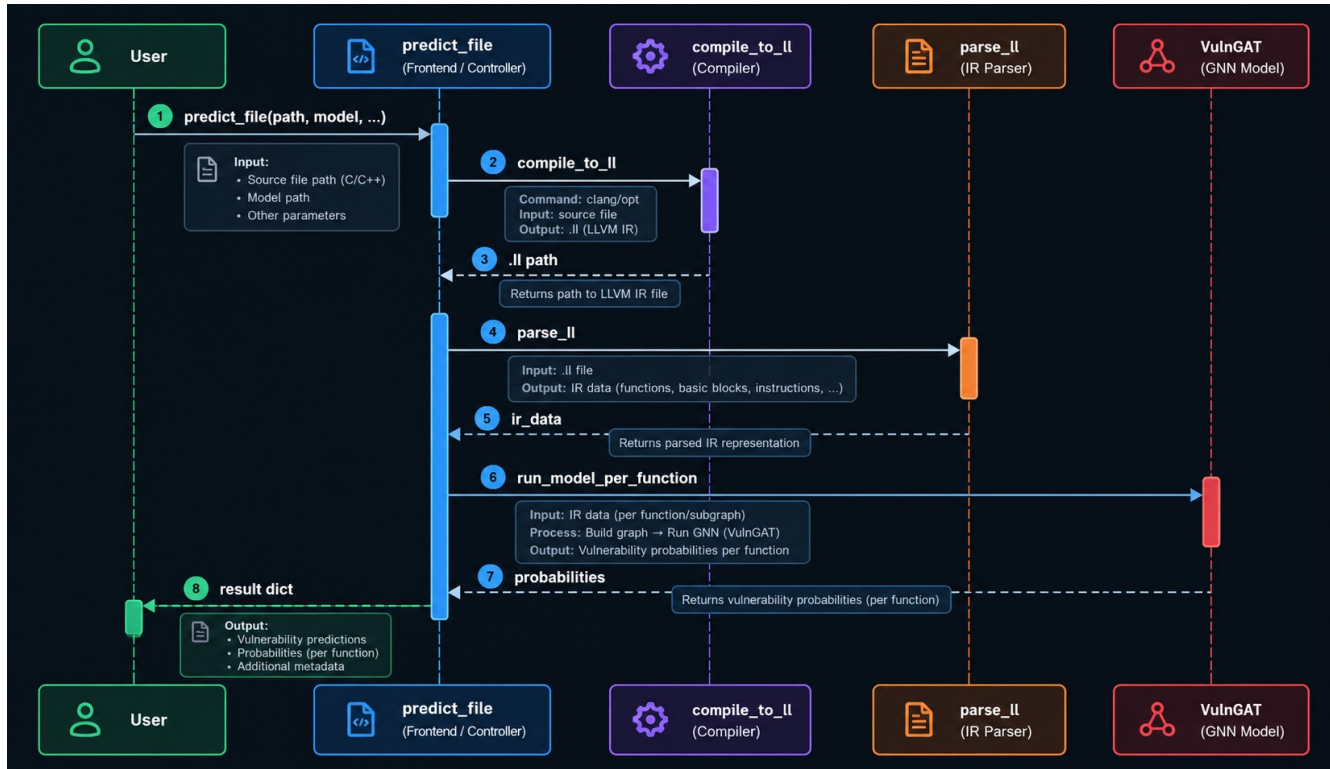


Figure 3. Supplied sequence diagram for `predict_file(path, model, ...)`.

→ The sequence shows the prediction orchestration: the user calls `predict_file`, compilation returns an LLVM IR path, parsing returns IR data, the model executes per function, probabilities return to the prediction layer, and a result dictionary is returned to the user.

Interaction	Meaning
<code>predict_file(path, model, ...)</code>	Top-level single-file inference entry point.
<code>compile_to_ir</code>	Transforms source into LLVM IR.
<code>.ll path</code>	Intermediate file used by subsequent parsing.
<code>parse_II</code>	Converts LLVM IR into structured data.
<code>ir_data</code>	Parsed representation passed to prediction logic.
<code>run_model_per_function</code>	Runs graph inference at function granularity.
<code>probabilities</code>	Model outputs after sigmoid/decision processing.
<code>result dict</code>	Structured final return value from <code>predict_file</code> .

6. Data Flow Diagram & Data Stores :-

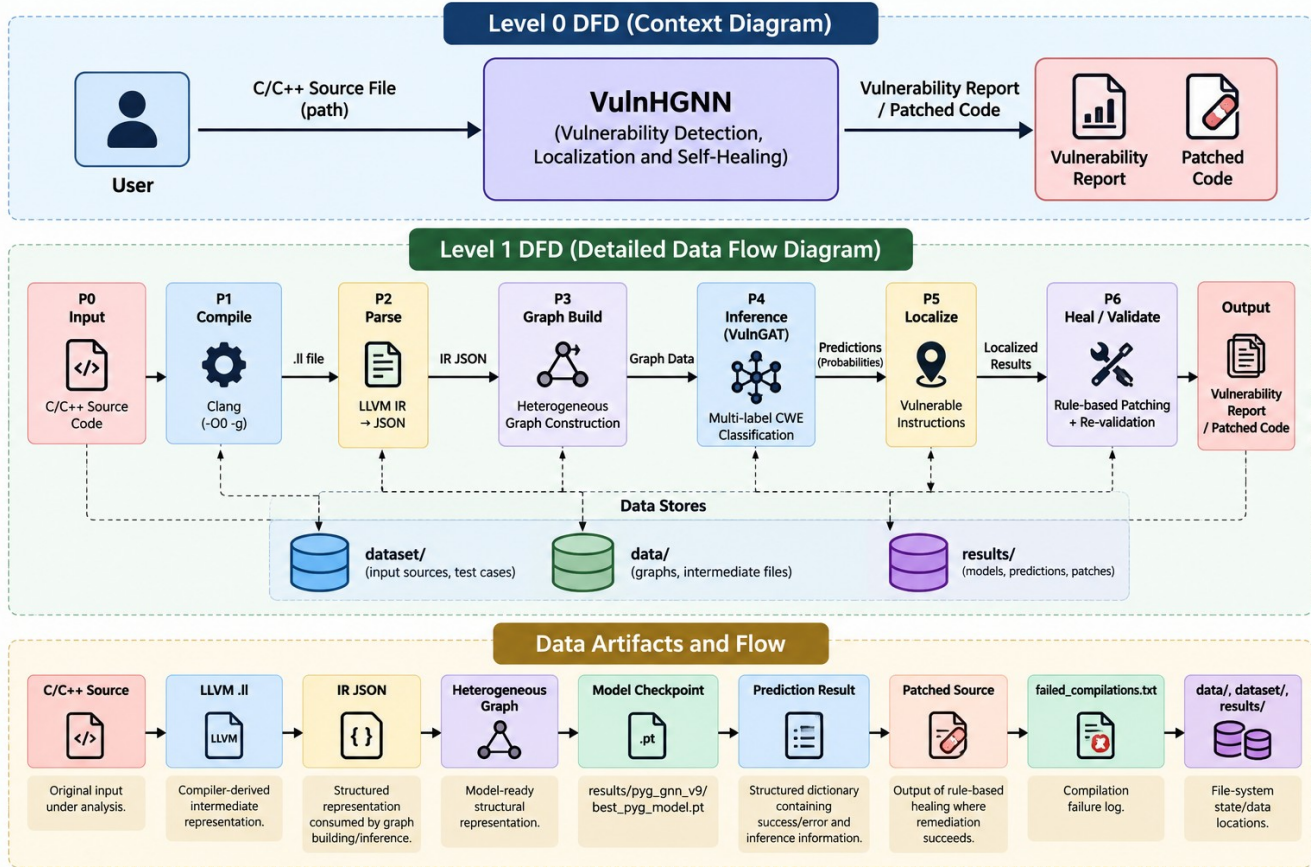


Figure 4. Data Flow & how Data Works / Stores in Project.

- **Data Flow Diagram :-**

1. Source code flows through compilation, parsing, graph construction, and inference.
2. Vulnerabilities are localized and processed for healing and validation.
3. The system produces a vulnerability report or patched source code.

7. Heterogeneous Graph Representation :-

7.1 Why a Heterogeneous Graph?

→ The project treats different program entities and relationships as distinct graph structure instead of flattening the source into a plain token sequence. This allows the downstream GNN to consume both node information and edge attributes while preserving structural context.

- **Heterogeneous Graph:-**

1. LLVM IR is transformed into a graph of instructions and basic blocks.
2. CFG, DFG, and containment edges preserve program structure and relationships.

3. The graph is converted into a PyTorch Geometric representation for GNN inference.

7.2 Diagram :-

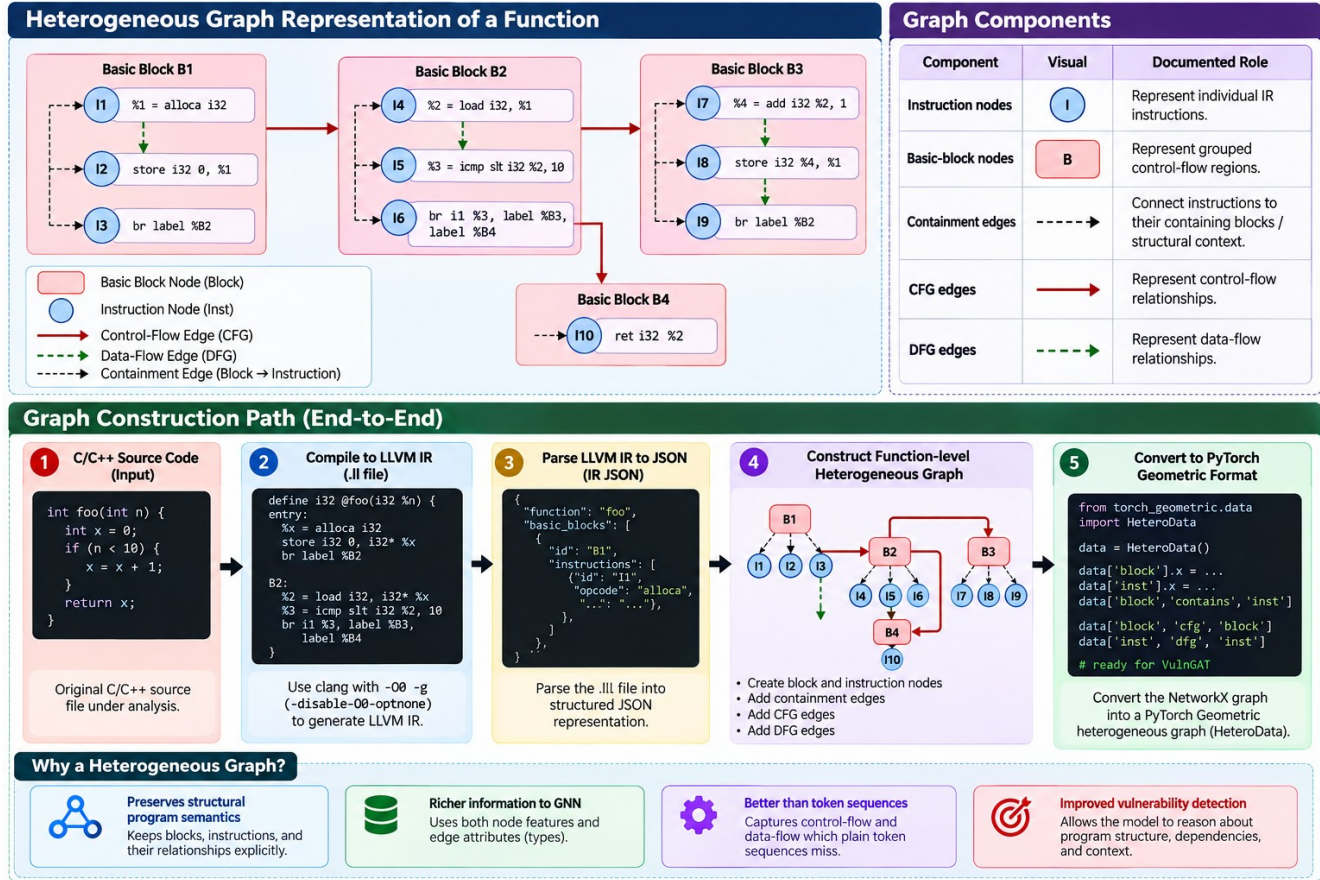


Figure 5. Construction of the Graph in from the llvm/ir Format.

8. VulnGAT AI Model Architecture :-

→ VulnGAT is documented as a multi label GNN implemented in `src/pyg_model.py` and used with PyTorch Geometric. The model receives node features, edge indices, edge attributes, and graph batch identifiers.

Model interface	Documented value
Model name	VulnGAT
Learning task	Multi label CWE classification
Framework	PyTorch + PyTorch Geometric
Forward inputs	<code>model(batch.x, batch.edge_index, batch.edge_attr, batch.batch)</code>
Output interpretation	Sigmoid probabilities per vulnerability class
Checkpoint	<code>results/pyg_gnn_v9/best_pyg_model.pt</code>
Known architecture detail	GAT style / edge attribute aware message passing
Not available in supplied index	Exact layer count, hidden dimensions, loss function, optimizer

9. File Level Decision Logic :-

9.1 Per Class Thresholding

Class	Threshold documented
CWE-476	0.35
CWE-191	0.50
CWE-190	0.50
CWE-369	0.50
non-vulnerable	0.50 (consistent with the documented 'others 0.5' rule)

9.2 Function to File Aggregation

→ For a file containing multiple analyzed functions, the documented file level score is the maximum probability across functions. Functions with fewer than three instructions are skipped in the documented inference flow.

9.3 Conceptual Decision Flow

Stage	Operation
Logits	Raw model outputs.
Sigmoid	Convert logits into per class probabilities.
Threshold	Compare each probability to its class specific cutoff.
Function verdict	Produce labels for each analyzed function.
File aggregation	Take the maximum function probability for each class.
Report	Return structured vulnerability findings.

10. Localization & Self Healing :-

10.1 Localization

→ Localization is positioned after model inference and before healing. The project analysis names src/localization.py as a known module, but states that the exact localization scoring equations are not present in the indexed content.

10.2 Healing

→ src/healer.py is referenced as the rule based remediation component. The architecture describes patches being applied and then re validated through a feedback loop. The supplied evidence does not expose the complete rule set or patch generation implementation, so this document describes the role rather than claiming specific repair rules.

10.3 Re validation

The `feedback_loop.py` component is referenced for re validation. Conceptually, a patch is not treated as the end of the pipeline: the modified program is analyzed again so that the system can report whether the vulnerability finding has been addressed.

10.4 Diagram

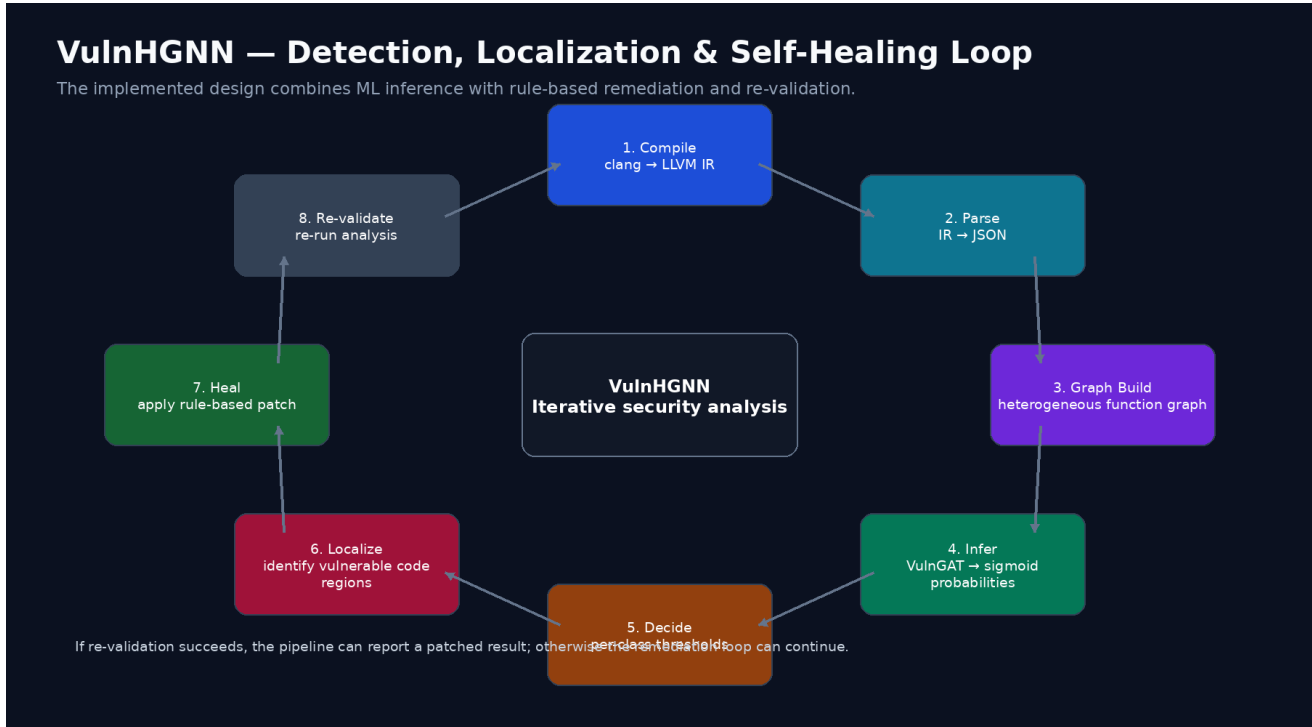


Figure 6. Newly generated detection → localization → healing → re-validation feedback loop.

11. Codebase Documentation :-

Module / File	Role
<code>src/compile.py</code>	clang → LLVM .ll compilation
<code>src/predict.py</code>	Single-file inference orchestration
<code>src/security.py</code>	IR JSON structural validation
<code>src/train_with_tests.py</code>	Training support + label verification
<code>src/pipeline.py</code>	Pipeline entry / orchestration
<code>src/pyg_model.py</code>	VulnGAT model
<code>src/localization.py</code>	Localization stage
<code>src/healer.py</code>	Rule based healing
<code>src/feedback_loop.py</code>	Re validation feedback loop
<code>src/app.py</code>	Flask web interface

12. Tech. Stack & Dependencies :-

Category	Technology	Documented purpose
Language	Python 3.8+	Application and ML pipeline
ML	PyTorch ≥ 2.0	Neural network computation
Graph ML	PyTorch Geometric	Graph batching / message passing
Graph	NetworkX	Graph construction
Embeddings / utilities	gensim	Project dependency
Data	NumPy, pandas	Numerical/data processing
Classical ML utilities	Scikit learn, scipy	Supporting processing/evaluation
Visualization	matplotlib	Visualization / project utilities
Progress	tqdm	Progress reporting
Web	Flask	Web UI
Emissions	CodeCarbon	Carbon/emissions tracking
Compiler	clang	C/C++ $\rightarrow$ LLVM IR
OS tooling	Windows .bat launchers	Documented launch method

13. Testing Features:-

verify_predictions acts as an end to end check over labeled target files and compares expected labels against predicted labels. The supplied analysis does not identify a formal pytest suite or published code coverage figures.

Testing layer
End to end label verification
Malformed IR validation

14. Installation & Setup :-

1. Create a virtual environment

```
python -m venv venv
```

2. Activate on Windows

```
venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Ensure clang is available on PATH

The compiler is required for LLVM IR generation.

5. Launch the Web UI

run.bat

6. CLI launch

run_pipeline.bat

7. Direct CLI invocation

```
set PYTHONPATH=%PYTHONPATH%;%CD%\src
```

```
python src\pipeline.py <file.c> --no-carbon --model results\pyg_gnn_v9\best_pyg_model.pt
```

Web endpoint documented by the project

<http://127.0.0.1:5000>

15. Limitations :-

Small Limitations :-

- Relatively small documented CWE class set.
- Functions with fewer than three instructions are skipped in the documented inference path, which can omit some real code regions.

Missing In Previous Code Base :-

1. Exact VulnGAT architecture details are not available in the supplied indexed context.
2. The full localization equations and healing rules are not available in the supplied indexed context.
3. No verified quantitative benchmark results are available in the supplied material.

Major Improvement (Try To Do) :-

- Due to Some Security Issues Instead Of Making The WebSite , Build the Secure Local CLI / Compiler .
- The Concept Of The **Obfuscation** to Secure the Code File From Hacker to retrieve .
- Adding the Agentic AI to project with the explainabilty called Explainable Intelligent AI **ExInt.AI** that Why this is wrong ? And How to Improve it ? , etc ...